

SCALA part2

>How to define **class** in scala much similar to java or c++

```
scala> class Ctr{
  | private var value =0 // fields must be initialised
  | def incr()={value+=1}
  | def curr()=value
  | }
class Ctr
```

In scala a class is not declared as public

A source file can contain multiple classes

All of the classes could be public

Previous class could be used in usual way

>object creation

```
val ctr=new Ctr
```

```
val ctr: Ctr = Ctr@43e30f15
```

```
ctr.inc()
```

Properties of getters and setters

Getter used to get the private variables

Setter to set private variables

```
scala> class Duck{
  | var size=1
  | }
class Duck
```

```
scala> val A=new Duck
```

```
val A: Duck = Duck@1a3e8b68
```

```
scala> A.size=(2)
```

```
// mutated A.size
```

```
scala> println(A.size)
```

```
2
```

>**getter ,setter** name should be same difference is _

```
class Duck{
  | private var privateAge=0
  | def age=privateAge
  | def age_=(newAge:Int){if(newAge>privateAge)privateAge=newAge;}
  | }
  def age_=(newAge:Int){if(newAge>privateAge)privateAge=newAge;}
    ^
```

On line 4: warning: procedure syntax is deprecated: instead, add `: Unit =` to explicitly declare `age\_\$eq`'s return type
class Duck

```
scala> val A=new Duck
val A: Duck = Duck@3b4a77be
```

```
scala> A.age=(2)
// mutated A.age
```

```
scala> print(A.age)
2
```

For the first case, declare the property as val scala treats it as final variable

In second case you need to declare field as private and provide the getter as explained below:

Semicolons are optional in scala

```
scala> class counter{
  | private var value=0
  | def inc()={value+=1}
  | def current=value}
class counter
```

```
scala> val w=new counter
val w: counter = counter@320d98f5
```

When we use setter for private val

```
class counter{
  | private val value=0
  | def inc()={value+=1}
  | def current=value
  | def current_=(newvalue:Int){if(newvalue>value)value=newvalue;}
  | }
  def current_=(newvalue:Int){if(newvalue>value)value=newvalue;}
                                ^
```

On line 5: warning: procedure syntax is deprecated: instead, add `: Unit =` to explicitly declare `current\_\$eq`'s return type

```
def inc()={value+=1}
          ^
```

On line 3: error: value += is not a member of Int

Expression does not convert to assignment because receiver is not assignable.

```
def current_=(newvalue:Int){if(newvalue>value)value=newvalue;}
                                ^
```

On line 5: **error: reassignment to val**

Auxiliary constructor

We can have many constructors as we need

There are two types of constructors in scala:

>>auxiliary constructor

>>primary constructor

The auxiliary constructor in scala are called **this**. This is different from the other languages,where constructor have the same class names the class

Each auxiliary constructor must start with a call to either a previously defined auxiliary constructors or primary constructor

```
scala> class duck{
  | private var size=0;
  | private var age=0;
  | def this(size:Int)={
  | this()
  | this.size=size}
  | def this(size:Int,age:Int)={
  | this(size) // calls the previous auxiliary constructor
  | this.age=age}
  | }
```

class duck

Here we cannot get as there is no getter

```
scala> class duck{
  | var size=0;
  | var age=0;
  | def this(size:Int)={
  | this()
  | this.size=size}
  | def this(size:Int,age:Int)={
  | this(size)
  | this.age=age}
  | }
```

class duck

```
scala> var cat=new duck(2,3)
var cat: duck = duck@6335c9e9
```

```
scala> var dog= new duck(5)
var dog: duck = duck@34a86c40
```

```
scala> var rat =new duck()
var rat: duck = duck@38c203a6
```

```
scala> println(cat.size,cat.age)
(2,3)
```

```
scala> println(rat.size, rat.age)
(0,0)
```

```
scala> println(dog.size, dog.age)
(5,0)
```

Primary constructor

-Every class in scala has a primary constructor

-primary constructor isn't defined by this method

The parameters for primary constructor are placed immediately after the class name:

```
scala> class duck(val size: Int, val age: Int){
  | def quack(): Unit =
  | {
  |   println("Quack")
  | }
  | def info(): String = {
  |   s"THIS duck is $size"
  | }
  | def growolder(): duck = {
  |   new duck(size, age+1)
  | }
class duck
```

```
scala> val dcuk1 = new duck(2, 1)
val dcuk1: duck = duck@121d434f
```

NESTED CLASSES

import scala.collection.mutable.ArrayBuffer

```
scala> class graph{
  | class member(val name: String)
  | {
  |   val contacts = new ArrayBuffer[member]
  | }
  | private val members = new ArrayBuffer[member]
  | def join(name: String) =
  | {
  |   val m = new member(name)
  |   members += m
  |   m
  | }
class graph
```

```
scala> val chatter=new graph
val chatter: graph = graph@544ec29e
```

```
scala> val mface=new graph
val mface: graph = graph@35b485aa
```

```
scala> val fred=chatter.join("fred")
val fred: chatter.member = graph$member@2e61ab3e
```

```
scala> val wilma=chatter.join("wilma")
val wilma: chatter.member = graph$member@24ea79c9
```

```
scala> fred.contacts+=wilma
val res18: fred.contacts.type = ArrayBuffer(graph$member@24ea79c9)
```

```
scala> val barney=mface.join("barney")
val barney: mface.member = graph$member@46a7a069
```

```
scala> fred.contacts+=barney
      ^
```

```
error: type mismatch;
 found   : mface.member
 required: chatter.member
```

Singletons

- >scala doesn't have the concept of static methods and static methods or fields
- it is supported by object construct
- >an object defines single instance of a class

```
scala> object reservation{
  | private var lastnum=0;
  | def newreservationnum()={lastnum+=1;lastnum}
  | }
object reservation
```

```
scala> reservation.newreservationnum()
val res20: Int = 1
```

```
scala> reservation.newreservationnum()
val res21: Int = 2
```

- If we need new reservation number,we call reservation.newreservationnum()
- constructor of singleton object is executed when the object is first used

Usecases

- when a singleton instance is required for co-ordinating

packages

-in scala packages serve the same purpose as in java :to manage the names in a large program

-to add the items to a package, they can be included in package statement

Example:

```
Package.com{  
  | Package.spark{  
  | Package prwatech{  
  | class emp{  
    .....  
  | }  
}
```

-This class emp can be accessed from anywhere as com.spark.prawatech.Emp

-Unlike,classes,a package can be defined in multiple files

-conversely a single file can have more than one package

Scope rules

–scope rules for package in scala are more consistent than java

-scala packages nest just like all other scopes

-member names coils be accessed from enclosing scope

```
Package com{  
  Package test1{  
    Object Utils{  
      Def sayHi="HI"  
    }  
    Package test2  
      {  
        Class Test  
        {def sayhello(){  
          utils.sayHi  
        }  
      }  
    }  
  }  
}
```

-note that we just used utils.sayHi,as object utils is defined in the parent package

Package visibility

–in java ,we typically control the class members by public,private or protected

- In scala the same effect could be achieved though qualifies

Import and impicit imports

-packages and classes can be imported in scala

-import serve the same purpose

- to use short names instead of long one

Polymorphism

Just like java classes can be extended using extend keyword

```
Class dog extends Animal{
```

```
    .....  
}
```

Example:

```
scala> class animal{  
    | println("Animals")}  
class animal
```

```
scala> class dog extends animal{  
    | println("dog")  
    | }  
class dog  
scala> val n =new dog  
Animals  
dog
```

We use final keyword to avoid inheritance

```
scala> final class animal{  
    | println("animal")}  
class animal
```

```
scala> class dog extends(animal){  
    | println("do")}  
    ^  
error: illegal inheritance from final class animal
```

-just like java methods and fields can be introduced

Or superclass methods or fields could be overridden in subclasses

-a class can be declared as final to avoid it being extended

Unlike

Overriding methods

-override modifier must be used to override an abstract method:

```
Public class dog{  
    .....  
    Override def bark={"woof"}  
    .....  
}
```

-the override modifier is useful in following scenarios:

-When name of the method being overridden is misspelled

-when a wrong parameter type is provided

-when a new method is introduced in superclass with clashes with a subclass method

- invoking super class method is same as in java ,by super keyword:

```

Public class Employee extends person{
    .....
    Override
}

```

Superclass constructor

-all the classes have a primary constructor and many auxiliary constructor ,and all the auxiliary constructor should either call a primary constructor or a previous constructor

It means an auxiliary constructor can never involve a superclass constructor

Abstract classes

-just like java you can use the abstract key word for a class ,which can't be instantiated:

```

Abstract class Emp(name:String){
    def id:Int//only method declaration ,no body
}

```

-here we declared a method to generate id,but didn't provide implementation

-each concrete subclass of emp should provide the implementation id

-In subclass,we don't need to specify override while defining an abstract superclass method:

class

```

Prawatech (name:String)extends Emp(name){
    Def id = name.hashCode//override keywor dnot required
}

```